

Detailed Report on Spring Actuator Features and Best Practices for Maintainable Applications

Introduction to Spring Actuator

Spring Actuator is a powerful tool provided by Spring Boot that helps developers monitor and manage Spring Boot applications. It provides production-ready features, such as metrics, health checks, and application insights, which are essential for maintainable and observable applications. Actuator simplifies the setup and configuration of monitoring systems, allowing for easier integration with external tools and dashboards.

Key Features of Spring Actuator

1. Health Checks:

- Actuator exposes a /health endpoint, which provides information on the overall health of the application.
- This includes:
 - Disk space
 - Database connectivity
 - External service availability
 - Custom health indicators

2. Metrics:

- Actuator's /metrics endpoint provides detailed information about the application's performance.
- Common metrics include:
 - Memory usage
 - CPU usage
 - HTTP request statistics
 - Thread counts
 - Custom business metrics

3. Application Info:

- The /info endpoint gives application metadata, such as:
 - Version number
 - Build information

- Descriptions about the application
- Developers can customize this endpoint by providing information in `application.properties` or `application.yml`.

4. **Tracing:**

- Provides tracing information related to request processing, helping identify performance bottlenecks.
- Can be integrated with distributed tracing systems like Zipkin or OpenTracing.

5. **Environment Properties:**

- Actuator exposes a `/env` endpoint, showing application environment details such as:
 - Configured properties
 - Active profiles
 - Environment variables

6. **Thread Dump:**

- The `/threaddump` endpoint provides a full thread dump, helping in debugging performance-related issues.

7. **Loggers:**

- The `/loggers` endpoint helps in dynamically changing log levels at runtime without redeploying the application.

8. **HTTP Trace:**

- The `/httptrace` endpoint displays the last few HTTP requests and responses made to the application.

9. **Auditing:**

- Actuator allows developers to record and expose auditing events via custom endpoints.

10. **Integration with External Monitoring Tools:**

- Spring Actuator integrates seamlessly with popular monitoring systems like:
 - Prometheus
 - Grafana
 - Datadog
 - New Relic
-

Custom Endpoints in Spring Actuator

Custom endpoints allow you to extend Actuator to expose custom application-specific information. This can be helpful for exposing specific data that is critical for the application's business logic, or to customize and secure existing endpoints.

Creating a Custom Endpoint

1. **Using the `@Endpoint` annotation:** You can create a custom endpoint by annotating a class with `@Endpoint`, and providing operations with methods annotated with `@ReadOperation`, `@WriteOperation`, or `@DeleteOperation` to handle GET, POST, and DELETE requests.

Example:

```
@Endpoint(id = "custom")
public class CustomEndpoint {
    @ReadOperation
    public String customStatus() {
        return "Custom Actuator Endpoint!";
    }
}
```

2. **Using the `@RestController` for Custom Actuator Endpoint:** You can create a custom REST endpoint in a Spring Boot application by using `@RestController` and map it with appropriate security levels for Actuator.

Example:

```
@RestController
public class CustomActuatorController {
    @GetMapping("/actuator/custom")
    public String customMetrics() {
        return "Custom Metrics";
    }
}
```

Allowing and Securing Endpoints

By default, Spring Actuator exposes some endpoints publicly and hides others. You can configure which endpoints are exposed and who can access them through security and property configurations.

Allowing/Enabling Endpoints

To enable or disable specific endpoints, configure the application's application.properties or application.yml file.

Example:

```
management.endpoints.web.exposure.include=health,info,metrics,custom
```

```
management.endpoint.custom.enabled=true
```

This will expose only the health, info, metrics, and custom endpoints.

Securing Endpoints

Securing sensitive endpoints like /env, /shutdown, and /loggers is crucial. This can be achieved by combining Spring Security with Actuator to restrict access based on user roles.

Example:

@Configuration

```
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {

        http
            .authorizeRequests()

            .requestMatchers(EndpointRequest.to(InfoEndpoint.class, HealthEndpoint.class)).permitAll()

            .requestMatchers(EndpointRequest.toAnyEndpoint()).hasRole("ADMIN")

            .and()

            .httpBasic();

    }
}
```

In the above example:

- The /info and /health endpoints are publicly accessible.
- All other Actuator endpoints are restricted to users with the ADMIN role.

HTTP vs. JMX Endpoints

- **HTTP Endpoints:** These are the default endpoints exposed over HTTP, e.g., /actuator/health. They are widely used for external monitoring.

- **JMX Endpoints:** For JVM-level monitoring, Actuator also supports JMX (Java Management Extensions). You can enable JMX endpoints for in-depth application management and performance diagnostics.

Example:

`management.endpoints.jmx.exposure.include=health,info`

Best Practices for Using Spring Actuator

1. Expose Only Necessary Endpoints:

- Don't expose all endpoints, especially sensitive ones like `/env` or `/shutdown` in production. Control exposure with the `management.endpoints.web.exposure.include` property.

2. Use Secure Channels:

- Always use HTTPS to secure the Actuator endpoints. This is especially important for sensitive endpoints like `/loggers` or `/jmx`.

3. Integrate with Monitoring and Alerting Tools:

- Leverage external monitoring and alerting tools (e.g., Prometheus, Grafana, New Relic) to visualize metrics and set up alerts based on health and performance data.

4. Limit Access to Authorized Users:

- Use Spring Security to ensure that only authorized users can access critical Actuator endpoints, such as `/metrics` or `/loggers`.

5. Monitor Custom Business Metrics:

- Extend Actuator to expose custom metrics or application-specific health checks. This allows better insights into business operations.

6. Profile-Specific Configurations:

- Customize which endpoints are available in different environments (e.g., development, testing, production). For instance, allow more endpoints during testing, but restrict them in production.

7. Log Important Events:

- Combine Actuator with effective logging practices. Use Actuator endpoints to monitor log levels and make sure logging configurations are optimal for performance.